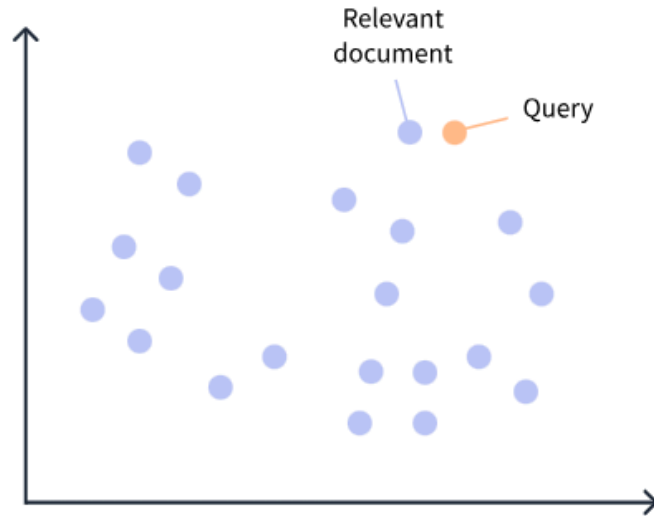


সেকশন 5-এ আমরা Datasets repository থেকে GitHub issues এবং comments নিয়ে একটি dataset তৈরি করেছি। এই সেকশনে আমরা সেই তথ্য ব্যবহার করে একটি search engine বানাব, যা library সম্পর্কে আমাদের গুরুত্বপূর্ণ প্রশ্নগুলোর উত্তর খুঁজে পেতে সাহায্য করবে।

Semantic search-এর জন্য embeddings ব্যবহার

চ্যাপ্টার 1-এ আমরা দেখেছিলাম, Transformer-ভিত্তিক language model কোনো text-এর প্রতিটি token-কে embedding vector আকারে প্রকাশ করে। দেখা যায়, এই আলাদা আলাদা embedding-গুলোকে একত্র করে পুরো sentence, paragraph, বা কিছু ক্ষেত্রে document-এর জন্যও একটি vector representation তৈরি করা যায়। এরপর প্রতিটি embedding-এর মধ্যে dot-product similarity (অথবা অন্য কোনো similarity metric) হিসাব করে corpus-এর মধ্যে মিল আছে এমন document খুঁজে বের করা যায়। যেসব document-এর overlap সবচেয়ে বেশি, সেগুলো return করা হয়।

এই সেকশনে আমরা embeddings ব্যবহার করে একটি semantic search engine তৈরি করব। এই ধরনের search engine-এর কিছু বাড়তি সুবিধা আছে, বিশেষ করে প্রচলিত সেইসব পদ্ধতির তুলনায়, যেখানে query-এর keyword-এর সাথে document-এর keyword মেলানো হয়।



ডেটাসেট লোড করা এবং প্রস্তুত করা

আমাদের প্রথম কাজ হলো GitHub issues-এর ডেটাসেটটি ডাউনলোড করা। তাই আগের মতোই `load_dataset()` ফাংশন ব্যবহার করি:

```
from datasets import load_dataset
```

```
issues_dataset = load_dataset("lewtun/github-issues", split="train")
issues_dataset
```

```
Dataset({
  features: ['url', 'repository_url', 'labels_url', 'comments_url', 'events_url', 'html_url', 'id',
'node_id', 'number', 'title', 'user', 'labels', 'state', 'locked', 'assignee', 'assignees', 'milestone',
'comments', 'created_at', 'updated_at', 'closed_at', 'author_association', 'active_lock_reason',
'pull_request', 'body', 'performed_via_github_app', 'is_pull_request'],
  num_rows: 2855
})
```

এখানে আমরা `load_dataset()`-এ ডিফল্ট `train` split নির্দিষ্ট করেছি, তাই এটি `DatasetDict` নয়, বরং একটি `Dataset` রিটার্ন করে। এখন প্রথম কাজ হলো pull request-গুলো বাদ দেওয়া, কারণ এগুলো সাধারণত user query-এর উত্তর দিতে খুব কম কাজে লাগে এবং আমাদের search engine-এ noise যোগ করে। এখন পর্যন্ত যেমন দেখেছি, `Dataset.filter()` ফাংশন ব্যবহার করে আমরা ডেটাসেট থেকে এসব row বাদ দিতে পারি। একই সঙ্গে যেসব row-এ কোনো comment নেই সেগুলোকেও বাদ দিই, কারণ এগুলো user query-এর কোনো উত্তর দেয় না।

```
issues_dataset = issues_dataset.filter(
  lambda x: (x["is_pull_request"] == False and len(x["comments"]) > 0)
)
issues_dataset
```

আমরা দেখতে পাচ্ছি, ডেটাসেটে অনেকগুলো column আছে, কিন্তু search engine বানানোর জন্য এগুলোর বেশিরভাগই আমাদের দরকার নেই। Search-এর দৃষ্টিকোণ থেকে সবচেয়ে তথ্যবহুল column হলো `title`, `body`, এবং `comments`; আর `html_url` আমাদের source issue-তে ফিরে যাওয়ার লিংক দেয়। তাই `Dataset.remove_columns()` ফাংশন ব্যবহার করে বাকি column-গুলো বাদ দিই:

```
columns = issues_dataset.column_names
columns_to_keep = ["title", "body", "html_url", "comments"]
columns_to_remove = set(columns).symmetric_difference(columns_to_keep)
issues_dataset = issues_dataset.remove_columns(columns_to_remove)
issues_dataset
```

```
Dataset({
  features: ['html_url', 'title', 'comments', 'body'],
  num_rows: 771
})
```

আমাদের embeddings তৈরির জন্য প্রতিটি comment-এর সঙ্গে issue-এর **title** এবং **body** যোগ করব, কারণ এই field-গুলোতে প্রায়ই দরকারি context থাকে। কিন্তু বর্তমানে **comments** column-এ প্রতিটি issue-এর জন্য comment-এর একটি list আছে। তাই আমাদের column-টি “explode” করতে হবে, যেন প্রতিটি row একটি **(html_url, title, body, comment)** tuple নিয়ে গঠিত হয়। Pandas-এ এটি **DataFrame.explode()** ফাংশন দিয়ে করা যায়। এটি list-জাতীয় column-এর প্রতিটি উপাদানের জন্য একটি নতুন row তৈরি করে এবং অন্য column-এর মানগুলো একইভাবে কপি করে রাখে। আগে এটি দেখার জন্য আমরা Pandas DataFrame format-এ যাই:

```
issues_dataset.set_format("pandas")
df = issues_dataset[:]
```

যদি আমরা DataFrame-এর প্রথম row দেখি, তাহলে বুঝতে পারব এই issue-এর সঙ্গে ৪টি comment যুক্ত আছে:

```
df["comments"][0].tolist()
```

এখন যখন আমরা **df**-কে explode করব, তখন আশা করি এই ৪টি comment-এর জন্য ৪টি আলাদা row পাব। দেখা যাক:

```
comments_df = df.explode("comments", ignore_index=True)
comments_df.head(4)
```

দারুণ, আমরা দেখতে পাচ্ছি row-গুলো replicate হয়েছে, আর **comments** column-এ এখন আলাদা আলাদা comment আছে। এখন Pandas-এর কাজ শেষ, তাই DataFrame-কে memory-তে load করে আবার দ্রুত **Dataset**-এ ফিরে যাই:

```
from datasets import Dataset

comments_dataset = Dataset.from_pandas(comments_df)
comments_dataset

Dataset({
  features: ['html_url', 'title', 'comments', 'body'],
  num_rows: 2842
})
```

```
}}
```

ঠিক আছে, এখন আমাদের কাজ করার জন্য কয়েক হাজার comment পাওয়া গেছে।

নিজে চেষ্টা করো! দেখো তুমি কি `issues_dataset`-এর `comments` column-কে `Dataset.map()` ব্যবহার করে explode করতে পারো, Pandas ব্যবহার না করে। এটি একটু কঠিন; এই কাজের জন্য Datasets documentation-এর “Batch mapping” অংশটি উপকারী হতে পারে।

এখন যেহেতু প্রতি row-তে একটি করে comment আছে, চল `comments_length` নামে একটি নতুন column তৈরি করি, যেখানে প্রতিটি comment-এ কতটি শব্দ আছে তা থাকবে:

```
comments_dataset = comments_dataset.map(
    lambda x: {"comment_length": len(x["comments"].split())}
)
```

এই নতুন column ব্যবহার করে আমরা ছোট comment-গুলো বাদ দিতে পারি। সাধারণত এর মধ্যে “cc @lewtun” বা “Thanks!”-এর মতো comment থাকে, যা আমাদের search engine-এর জন্য তেমন কাজে লাগে না। Filter-এর জন্য নির্দিষ্ট কোনো সঠিক সংখ্যা নেই, তবে প্রায় ১৫টি শব্দ একটি ভালো শুরু হতে পারে:

```
comments_dataset = comments_dataset.filter(lambda x: x["comment_length"] > 15)
comments_dataset
```

```
Dataset({
  features: ['html_url', 'title', 'comments', 'body', 'comment_length'],
  num_rows: 2098
})
```

আমাদের ডেটাসেট কিছুটা পরিষ্কার করার পর, এখন issue-এর `title`, `body`, এবং `comments` একসঙ্গে জুড়ে `text` নামে একটি নতুন column বানাই। আগের মতোই, এ জন্য একটি ছোট ফাংশন লিখব যা `Dataset.map()`-এ পাঠানো যাবে:

```
def concatenate_text(examples):
    return {
        "text": examples["title"]
        + "\n "
        + examples["body"]
        + "\n "
        + examples["comments"]
    }
```

```
comments_dataset = comments_dataset.map(concatenate_text)
```

টেক্সট এমবেডিং তৈরি করা

আমরা Chapter 2-এ দেখেছিলাম যে `AutoModel` class ব্যবহার করে token embeddings পাওয়া যায়। এর জন্য আমাদের শুধু model load করার উপযোগী একটি checkpoint বেছে নিতে হয়। সৌভাগ্যবশত, `sentence-transformers` নামে একটি library আছে, যা বিশেষভাবে embeddings তৈরির জন্য বানানো হয়েছে। Library-টির documentation-এ যেমন বলা হয়েছে, আমাদের use case হলো **asymmetric semantic search**-এর একটি উদাহরণ, কারণ আমাদের কাছে একটি ছোট query আছে, যার উত্তর আমরা একটি বড় document-এর মধ্যে খুঁজতে চাই, যেমন একটি issue comment-এর মধ্যে।

Documentation-এর সুবিধাজনক model overview table থেকে দেখা যায় যে `multi-qa-mpnet-base-dot-v1` checkpoint semantic search-এর জন্য সবচেয়ে ভালো performance দেয়। তাই আমরা আমাদের application-এর জন্য এটি ব্যবহার করব। একই checkpoint ব্যবহার করে tokenizer-টিও load করব:

```
from transformers import AutoTokenizer, AutoModel

model_ckpt = "sentence-transformers/multi-qa-mpnet-base-dot-v1"
tokenizer = AutoTokenizer.from_pretrained(model_ckpt)
model = AutoModel.from_pretrained(model_ckpt)
```

Embedding process দ্রুত করার জন্য model এবং inputs-কে GPU device-এ রাখা ভালো, তাই এখন সেটি করি:

```
import torch

device = torch.device("cuda")
model.to(device)
```

আগেই বলেছি, আমরা চাই আমাদের GitHub issues corpus-এর প্রতিটি entry-কে একটি single vector হিসেবে প্রকাশ করতে। তাই token embeddings-গুলোকে কোনোভাবে “pool” বা average করতে হবে। একটি জনপ্রিয় পদ্ধতি হলো model-এর outputs-এর উপর **CLS pooling** করা, যেখানে আমরা special `[CLS]` token-এর জন্য last hidden state সংগ্রহ করি। নিচের function-টি আমাদের জন্য সেই কাজটি করে:

```
def cls_pooling(model_output):
    return model_output.last_hidden_state[:, 0]
```

এবার আমরা একটি helper function তৈরি করব, যা documents-এর একটি list tokenize করবে, tensor-গুলোকে GPU-তে পাঠাবে, সেগুলো model-এ feed করবে, এবং শেষে outputs-এর উপর CLS pooling প্রয়োগ করবে:

```
def get_embeddings(text_list):
    encoded_input = tokenizer(
        text_list, padding=True, truncation=True, return_tensors="pt"
    )
    encoded_input = {k: v.to(device) for k, v in encoded_input.items()}
    model_output = model(**encoded_input)
    return cls_pooling(model_output)
```

Function-টি ঠিকভাবে কাজ করছে কি না, তা পরীক্ষা করার জন্য আমরা corpus-এর প্রথম text entry-টি এতে দেব এবং output-এর shape দেখব:

```
embedding = get_embeddings(comments_dataset["text"][0])
embedding.shape

torch.Size([1, 768])
```

দারুণ, আমরা আমাদের corpus-এর প্রথম entry-টিকে 768-dimensional vector-এ রূপান্তর করেছি! এখন `Dataset.map()` ব্যবহার করে `get_embeddings()` function-টি corpus-এর প্রতিটি row-তে প্রয়োগ করতে পারি। তাই নিচের মতো করে একটি নতুন `embeddings` column তৈরি করি:

```
embeddings_dataset = comments_dataset.map(
    lambda x: {"embeddings": get_embeddings(x["text"]).detach().cpu().numpy()[0]}
)
```

দ্রুত **similarity search**-এর জন্য **FAISS** ব্যবহার

এখন যেহেতু আমাদের কাছে embeddings-এর একটি dataset আছে, তাই এগুলোর উপর search চালানোর জন্য কোনো উপায় দরকার। এই কাজের জন্য আমরা Datasets-এর একটি বিশেষ data structure ব্যবহার করব, যার

নাম **FAISS index**।

FAISS-এর পূর্ণরূপ হলো **Facebook AI Similarity Search**। এটি এমন একটি library, যা embedding vector-গুলোর উপর দ্রুত search এবং clustering করার জন্য কার্যকর algorithm দেয়।

FAISS-এর মূল ধারণা হলো একটি বিশেষ data structure তৈরি করা, যাকে **index** বলা হয়। এই index আমাদের সাহায্য করে বুঝতে, কোন embedding-গুলো একটি input embedding-এর সাথে সবচেয়ে বেশি মিল রয়েছে।

Datasets-এ FAISS index তৈরি করা খুবই সহজ — আমরা `Dataset.add_faiss_index()` ফাংশন ব্যবহার করি এবং dataset-এর কোন column-টিকে index করতে চাই তা নির্দিষ্ট করে দিই:

```
embeddings_dataset.add_faiss_index(column="embeddings")
```

এখন আমরা `Dataset.get_nearest_examples()` ফাংশন ব্যবহার করে এই index-এর উপর query চালাতে পারি। এটি nearest neighbor lookup করে। আগে একটি প্রশ্নকে embedding-এ রূপান্তর করে দেখি:

```
question = "How can I load a dataset offline?"
question_embedding = get_embeddings([question]).cpu().detach().numpy()
question_embedding.shape
```

```
torch.Size([1, 768])
```

Document-গুলোর মতোই এখন আমাদের query-টিও একটি 768-dimensional vector-এ পরিণত হয়েছে। এখন এটিকে পুরো corpus-এর সাথে তুলনা করে সবচেয়ে মিল আছে এমন embeddings খুঁজে বের করা যাবে:

```
scores, samples = embeddings_dataset.get_nearest_examples(
    "embeddings", question_embedding, k=5
)
```

`Dataset.get_nearest_examples()` ফাংশন দুটি জিনিস return করে:

- **scores**: query এবং document-এর মধ্যে কতটা মিল আছে তার মান
 - **samples**: সেই অনুযায়ী পাওয়া document/sample-গুলো
- এখানে আমরা সবচেয়ে ভালো ৫টি match চেয়েছি।

এখন এগুলোকে একটি `pandas.DataFrame`-এ রাখি, যাতে সহজে sort করা যায়:

```
import pandas as pd

samples_df = pd.DataFrame.from_dict(samples)
samples_df["scores"] = scores
samples_df.sort_values("scores", ascending=False, inplace=True)
```

এখন আমরা প্রথম কয়েকটি row ঘুরে দেখে নিতে পারি, আমাদের query উপলব্ধ comment-গুলোর সাথে কতটা ভালোভাবে match করেছে:

```
for _, row in samples_df.iterrows():  
    print(f"COMMENT: {row.comments}")  
    print(f"SCORE: {row.scores}")  
    print(f"TITLE: {row.title}")  
    print(f"URL: {row.html_url}")  
    print("=" * 50)  
    print()
```